Datenstrukturen: Bäume, Graphen und Heaps



Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 2.1.0.2

Quelle: Die Folien sind teilweise inspiriert von oder basierend auf: Lehrmaterial von Prof. Dr. Scherer, Prof. Dr. Baumgart

---

Kontrollfragen: https://delors.github.io/theo-ds-baeume-und-graphen/kontrollfragen.de.md.html

Folien: https://delors.github.io/theo-ds-baeume-und-graphen/folien.de.md.html
https://delors.github.io/theo-ds-baeume-und-graphen/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Graphen

Gerichteter Graph

Definition

Ein gerichteter Graph $G = (V, E)$ wird durch eine Knotenmenge V und eine Kantenmenge E beschrieben.

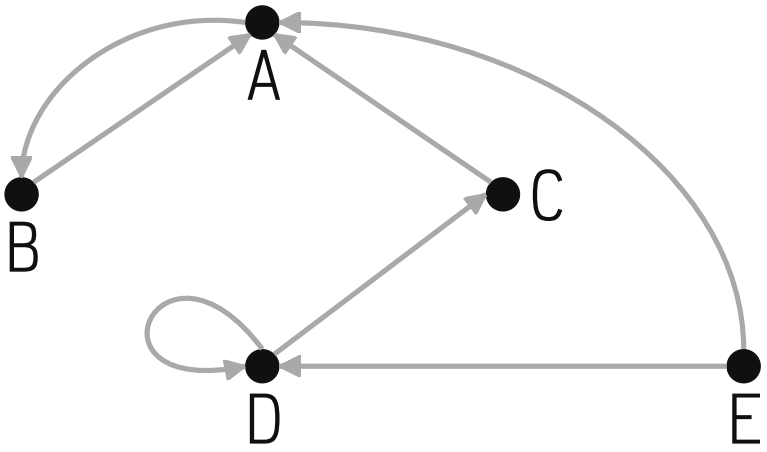
$V = \{v_1, v_2, ..., v_n\}$

$E = \{(v_i, v_j) | v_i, v_j \in V\}$

Bemerkung

- Es ist erlaubt, dass ein Knoten (Vertex) eine Kante (Edge) zu sich selbst besitzt
- Da E eine Menge ist, kann es keine doppelten Elemente geben; folglich gilt:
- Es gibt maximal zwei Kanten zwischen zwei Knoten vi und vj (i≠ j)
- Ein Knoten kann maximal eine Kante zu sich selbst haben
- Statt $v_1, ...v_n$ werden häufig (Groß-) Buchstaben A, B, C etc. verwendet

Gerichteter Graph - Beispiel



Ungerichteter Graph

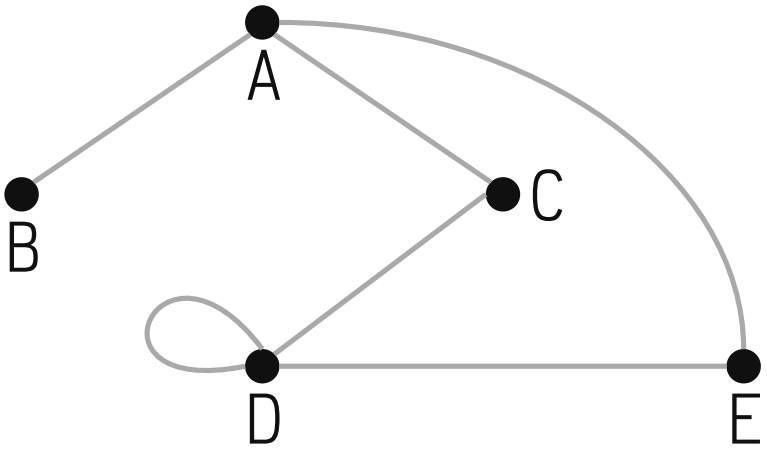
Definition

Ein ungerichteter Graph $G = (V, E)$ ist ein gerichteter Graph, in dem die Relation E symmetrisch ist:

$(v_i, v_j) \in E \Rightarrow (v_j, v_i) \in E$

Die Kanten $\{(v_i, v_j), (v_j, v_i)\}$ werden dann als eine Kante gezählt.

Ungerichteter Graph - Beispiel



Gewichteter Graph

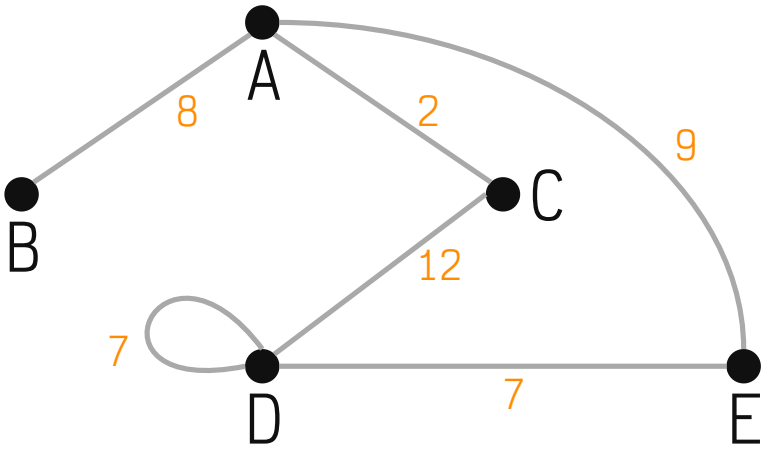
Definition

Bei einem gewichteten Graphen $G = (V, E)$ besitzt jede Kante (v_i, v_j) eine Attributierung $w(v_i, v_j)$.

Bemerkung

- Sowohl gerichtete als auch ungerichtete Graphen können gewichtet sein
- Die Funktion w hat als Wertebereich oft $\mathbb{N}$, $\mathbb{Z}$ oder $\mathbb{R}$
- Die Attributierung kann aber auch aus mehreren Attributen bestehen, z. B. aus einer Zahl und einer (Kanten-) Farbe; für dieses Beispiel sind die Ergebnisse von w dann Zweitupel

Gewichteter Graph - Beispiel



Markierte Graphen

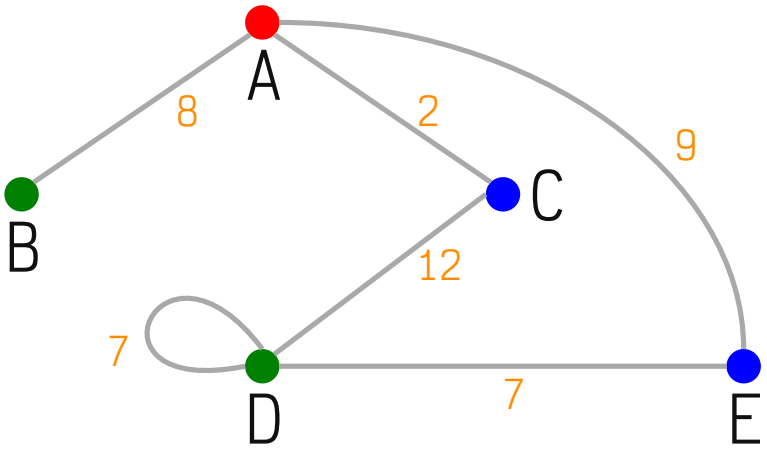
Definition

In einem markierten Graphen $G = (V, E)$ besitzt jeder Knoten v_i eine Markierung $m(v_i)$.

Bemerkung

- Die Markierung stellt eine Attributierung des Knotens dar
- Die Markierung ist häufig ein Farbwert
- Alle Typen von Graphen (gerichtet, ungerichtet, gewichtet) können markiert werden
- Die Markierung spielt in vielen Graphenalgorithmen eine Rolle (z. B. bei der Breite-zuerst-Suche)

Markierter (gewichteter) Graph - Beispiel



Pfad

Definition

Ein Pfad in einem Graphen $G = (V, E)$ mit $V = \{v_1, ..., v_n\}$ ist eine Folge von Knoten $v_{a_1}, v_{a_2}, ..., v_{a_k}$ mit folgender Eigenschaft: $(v_{a_i}, v_{a_{i+1}}) \in E$ für $i \in \{1, ..., k-1\}$.

Die Länge des Pfades ist die Anzahl der Kanten auf dem Pfad, d. h. $k - 1$.

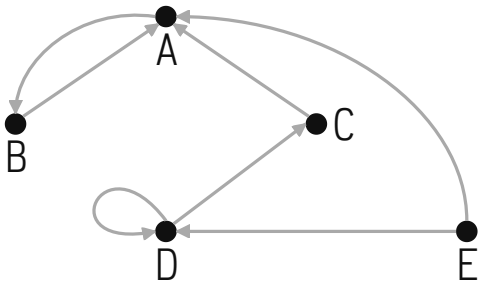
Bemerkung

- In einem Pfad dürfen Knoten mehrfach auftreten
- Für Pfade wird häufig eine Listenschreibweise verwendet: $< v3, v1, v2, v3, v4 >$

Fragestellungen auf Graphen

- Welche Knoten sind von einem (Start-) Knoten aus erreichbar? → Erreichbarkeitsbaum
- Existiert zwischen 2 Knoten A und B ein Pfad, d. h. ist B von A aus erreichbar?
- Wenn mehrere Pfade zwischen 2 Knoten existieren, welcher ist der kürzeste?
- Existieren in einem Graphen Zyklen?

Implementierung von Graphen



Adjazenzliste

Struktur der Adjazenzlisten für einen Graphen $G = (V, E)$:

Es gibt eine Knotenliste L_0, die alle Knoten aus V enthält.

Jedem Element $v_i (i \in v_1, ..., v_n)$ der Knotenliste L_0 ist eine weitere Knotenliste L_i zugeordnet, die die Endknoten der Kanten aus E enthält, die von v_i ausgehen

Wir speichern einen Graphen somit als Liste von Listen.

Beispiel

Seien A bis E Instanzen der Klasse Node, die einen Knoten im Graphen repräsentieren.

```
// Konzeptionell
// L0 = List<Tuple<Node,List<Node>>>(
L0 = Map<Node,List<Node>>.of(
    A, List.of(B),
    B, List.of(A),
    C, List.of(A),
    D, List.of(D,C),
    E, List.of(D,A),
)
```

Bewertung

★ Kompaktere Speicherung

★ Erweiterbarkeit

✗ Aufwändigerer Zugriff z. B. bei der Adressierung der Kanten

✗ Höhere Komplexität für die Beantwortung von Fragen wie

- Existiert eine Kante von x nach y?
- Welche Kanten enden an einem Knoten x?

Adjazenzmatrix

Struktur der Adjazenzmatrix für einen Graphen $G = (V, E)$:

- Die Zeilen und Spalten der Adjazenzmatrix sind mit der Knotenmenge V beschriftet
- Die Matrixelemente dienen zur Darstellung der Kantenrelation E eines Graphen G = (V, E)

AJ[x, y] = 1 ⇔ es existiert eine gerichtete Kante von x nach y

- Bei einem gewichteten Graphen enthält die Adjazenzmatrix die Kantengewichte w(x, y)

Beispiel

	A	B	C	D	E
A	0	1	0	0	0
B	1	0	0	0	0
C	1	0	0	0	0
D	0	0	1	1	0
E	1	0	0	1	0

Bewertung

- ★ Extrem schnelle und einfache Adressierbarkeit der Kanten
- ★ Semantik einer Zeile: Welche Kanten gehen von dem zugehörigen Knoten weg
- ★ Semantik einer Spalte: Welche Kanten enden an dem zugehörigen Knoten
- ✗ Bei wenigen Kanten ist die Matrix nur dünn besetzt → Speicherplatzverschwendung
- ✗ Arrays als typische Implementierung für Matrizen erfordern Reorganisationsaufwand, wenn dem Graphen neue Knoten hinzugefügt werden

Suchalgorithmen auf Graphen

Suchalgorithmen auf Graphen - Grundlegendes

Eingaben

- (Gerichteter) Graph $G = (V, E)$
- Startknoten v_s

Ablauf

- S1: Markiere Startknoten v_s
- S2: Wähle Kante (v_i, v_j) mit folgenden Eigenschaften:

  v_i ist markiert und v_j ist nicht markiert
- S3: Markiere v_j und setze mit Schritt S2 fort

Endebedingung

Es existiert in Schritt S2 kein unmarkierter Knoten mehr

Ergebnis

Genau ein Erreichbarkeitsbaum wird markiert.

Wichtige Varianten

- Breite-zuerst-Suche
- Tiefe-zuerst-Suche

(Die Unterscheidung erfolgt im Schritt S2 in Hinblick auf die auszuwählende Kante.)

Benötigte Konzepte

- Aktiver und passiver Knoten

Kennzeichnung eines Knotens als aktiv, wenn von ihm aus die Suche (gegebenenfalls) später fortgesetzt werden kann.

- allg. Vorgehensweise
- Kennzeichnung eines neu markierten Knotens als aktiv
- Ein aktiver Knoten v wird passiv, wenn gilt:

  Alle von v ausgehenden Kanten wurden dahingehend untersucht, ob sie zu einem noch nichtmarkierten Knoten führen.

Ergebnis

- Weitersuche ist nur von aktiven Knoten aus sinnvoll
- Nur markierte Knoten können aktiv sein

Breite-zuerst-Suche (🇺🇸 *Breadth-first-Search*)

Im Schritt S2

- Durchsuchung aller vom Suchknoten ausgehenden Kanten dahingehend, ob sie zu einem (bisher) unmarkierten Knoten führen.
- Markierung eines derartigen Knotens.
- Aufnahme des Knotens in die Liste aktiver Knoten.
- Gibt es keine derartige Kante mehr, wähle als neuen Suchknoten den ältesten noch aktiven Knoten.

  (Als Datenstruktur dient die Verwendung einer FIFO-Warteschlange)

Endekriterium

Vom Suchknoten aus wird kein unmarkierter Nachfolgerknoten mehr gefunden und die Liste aktiver Knoten ist leer.

!! Wichtig

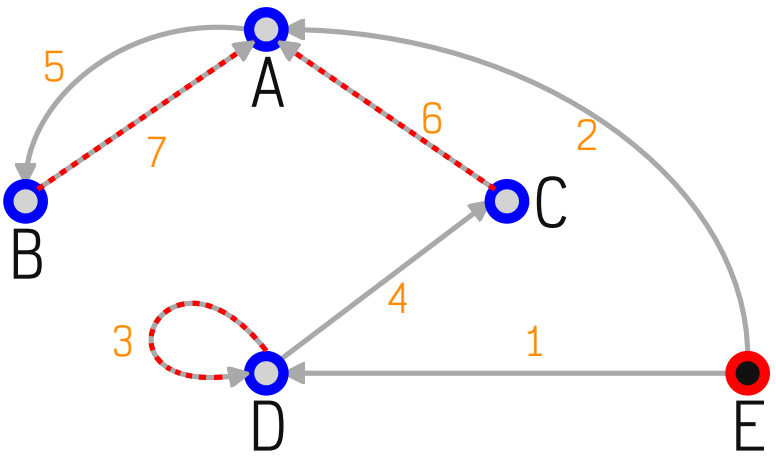
Unter allen möglichen Erreichbarkeitsbäumen wird derjenige mit den kürzesten Pfaden erzeugt.

Dies liegt an der Struktur der Suchsteuerung in S2:

- Inkrementeller Aufbau der Pfade
- Weiterhin werden die Pfade nicht sequentiell, sondern simultan aufgebaut

⇒ Knoten werden markiert, bevor lange Pfade entstehen können

Breite-zuerst-Suche - Beispiel



Tiefe-zuerst-Suche

Im Schritt S2

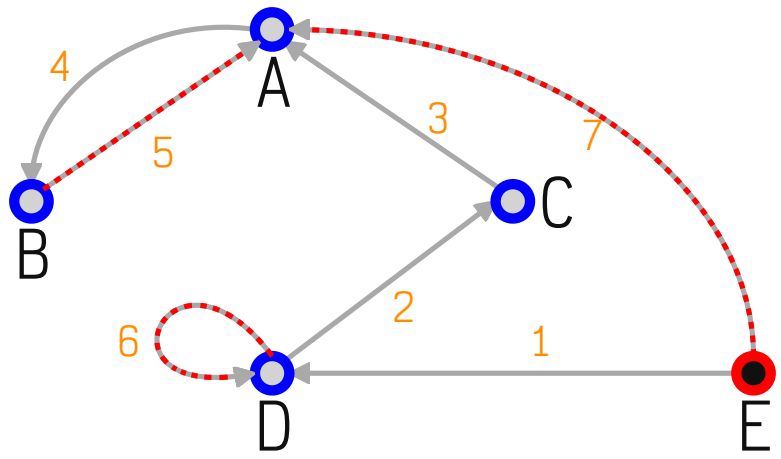
- Suche vom aktuellen Suchknoten nd eine Kante zu einem (bisher) unmarkierten Knoten nd_{neu}
- Markierung von nd_{neu}
- Aufnahme des Knotens nd (nicht nd_{neu}!) in die Liste aktiver Knoten
- nd_{neu} wird zum neuen Suchknoten und die Suche wird mit nd_{neu} fortgesetzt
- Gibt es keine derartige Kante mehr, wähle als neuen Suchknoten den jüngsten noch aktiven Knoten

  (Zu verwendende Datenstruktur: Stack (LiFO))

Endekriterium

Vom Suchknoten aus wird kein unmarkierter Nachfolgerknoten mehr gefunden und die Liste aktiver Knoten ist leer.

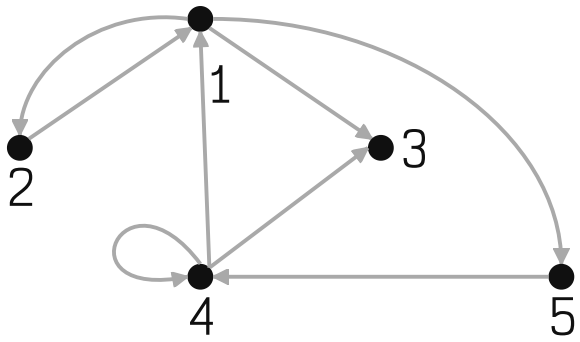
Tiefe-zuerst-Suche - Beispiel



✎ Übung

1.1. Tiefe-/Breite-zuerst-Suche

Gegeben sei der folgende Graph:



Führen Sie sowohl eine Tiefe- als auch Breite-zuerst-Suche ausgehend von Knoten 4 als auch Knoten 5 durch. Sollte es mehrere Möglichkeiten geben, dann wählen Sie zuerst den Knoten mit der kleineren Nummer!

2. Bäume

Bäume - Einführung

- Bäume sind eine Datenstruktur für hierarchische Daten

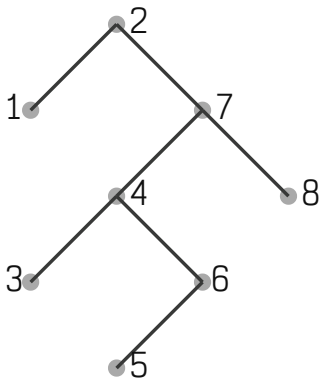
  ### Beispiele

  - Stammbaum
  - Dateisystem
  - Organisationsstruktur

- relevante Operationen
  - Schnelles Abändern
  - Schnelles Suchen

Bäume - Schlüsselbegriffe

Beispielbaum



Elternknoten (🇺🇸 *Parent*)

Ein Elternknoten ist ein Knoten, der einen oder mehrere Kinder hat.

Beispiel

2 ist der Elternknoten von 7.

4 ist der Elternknoten von 3.

...

Kindknoten (🇺🇸 *Child(ren)*)

Kindknoten sind Knoten, die einen Elternknoten haben.

Beispiel

1 und 7 sind Kindknoten von 2.

5 ist Kindknoten von 6.

...

Wurzel (🇺🇸 *Root*)

Der Wurzelknoten ist der einzige Knoten, der keinen Elternknoten hat.

Beispiel

2 ist die Wurzel/der Wurzelknoten.

Blattknoten (🇺🇸 *Leaf*)

Ein Blattknoten ist ein Knoten, der keine Kinder hat.

Beispiel

1, 8, 3, 5 sind Blattknoten.

Innerer Knoten (🇺🇸 *Inner Node*)

Ein Knoten, der kein Blattknoten ist, ist ein innerer Knoten.

Beispiel

4, 7 und 6 sind innere Knoten. Auch der Wurzelknoten ist ein innerer Knoten, wenn der Baum nicht degeneriert ist.

Höhe eines Baumes (🇺🇸 *Height*)

Die Höhe eines Baumes ist die Länge des längsten Pfades vom Wurzelknoten zu einem Blattknoten.

Tiefe eines Knotens (🇺🇸 *Depth*)

Die Tiefe eines Knotens p ist die Länge des Pfades vom Wurzelknoten zu p.

Definition: Binärbaum

Ein Baum, bei dem jeder Knoten höchstens zwei Kinder hat.

Definition: Binärer Suchbaum

Ein Binärbaum, bei dem für jeden Knoten gilt: Alle Schlüssel in den linken Unterbaum sind kleiner und alle Schlüssel in den rechten Unterbaum sind größer.

Beide Unterbäume sind wiederum binäre Suchbäume (rekursive Definition).

Optional kann jeder Knoten neben dem Schlüssel auch weitere Nutzdaten speichern.

Beispiel

(siehe Darstellung)

Achtung!

Da die Kanten in einem Baum grundsätzlich von einem Eltern- zu einem Kindknoten gehen (d. h. es sind gerichtete Kanten), wird die Kantenrichtung typischerweise nicht angegeben (d. h. es wird keine Pfeilspitze gezeichnet).

Traversierung von Bäumen

Traversierung von Bäumen beschreibt die Reihenfolge, in der Knoten eines Baumes besucht werden.

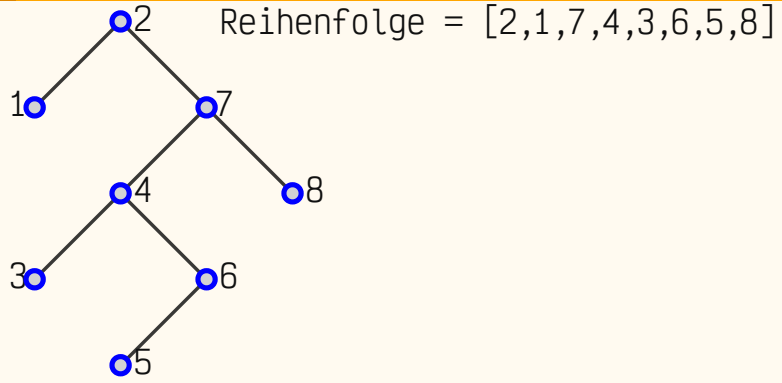
Wir können drei Formen von Traversierung unterscheiden:

1. Präorder
2. Inorder und
3. Postorder

Traversierung.

Präorder Traversierung (🇺🇸 *Preorder Traversal*)

Beispiel



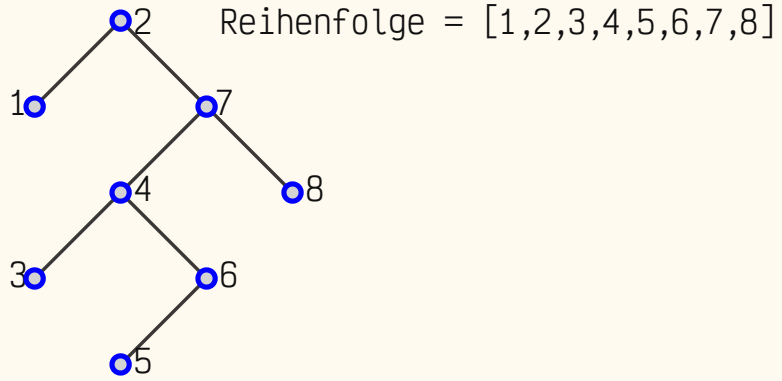

```
Reihenfolge = [2,1,7,4,3,6,5,8]
```

Definition

Eine Präorder Traversierung eines (Teil-)Baumes beginnt mit der Verarbeitung des aktuellen (Wurzel-)Knotens, danach besuchen wir rekursiv den linken Unterbaum, bevor der rechte Unterbaum besucht wird.

Inorder Traversierung (🇺🇸 *Inorder Traversal*)

Beispiel




```
Reihenfolge = [1,2,3,4,5,6,7,8]
```

Definition

Eine Inorder-Traversierung eines Baumes beginnt mit dem linken Unterbaum, besucht dann den (Wurzel-)knoten, bevor der rechte Unterbaum besucht wird.

Bemerkung

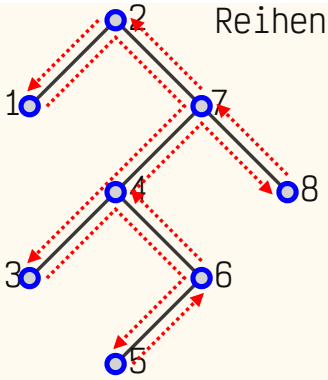
Eine Inorder-Traversierung eines (binären) Suchbaums führt zu einer sortierten Liste der Knotenwerte. Es handelt sich somit um ein weiteres Beispiel für ein Sortierverfahren.

Postorder-Traversierung (🇺🇸 *Postorder Traversal*)

Beispiel

Definition

Eine Postorder-Traversierung eines Baumes beginnt mit dem linken Unterbaum und besucht dann den rechten Unterbaum, bevor der




```
Reihenfolge = [1,3,5,6,4,8,7,2]
```

(Wurzel-)knoten besucht wird.

Suche in einem binären Suchbaum (BST)

Ziel: Finde Schlüssel `x`

1. Setze den aktuellen Knoten `n` auf den Wurzelknoten des Baums.
2. Falls der aktuelle Knoten `n` „leer“ ist, dann ist `x` nicht enthalten im Baum.
3. Falls `x == key(n)` : Der Schlüssel wurde gefunden.
4. Falls `x < key(n)` : Setze `n` auf den Wurzelknoten des linken Unterbaums.
5. Sonst: Setze `n` auf den Wurzelknoten des rechten Unterbaums.
6. Gehe zu Schritt 2.

Laufzeit: proportional zur Höhe `h` des Baums, also `O(h)` .

Implementierung eines binären Suchbaums (BST)

```java
// BST.java

import java.util.Deque;
import java.util.LinkedList;
import java.util.Queue;

public class BST<Key extends Comparable<Key>, Value> {

    private class Node {
        private Key key;
        private Value val;
        private Node left, right;
        private int count;

        public Node(Key key, Value val) {
                this.key = key;
                this.val = val;
                this.count = 1;
        }
    }

    private Node root;

    public void put(Key key, Value val) {
        root = put(root, key, val);
    }

    private Node put(Node x, Key key, Value val) {
        if (x == null)
                return new Node(key, val);
        int cmp = key.compareTo(x.key);
        if (cmp < 0)
                x.left = put(x.left, key, val);
        else if (cmp > 0)
                x.right = put(x.right, key, val);
        else if (cmp == 0)
                x.val = val;
        x.count = 1 + size(x.left) + size(x.right);

        return x;
    }

    public Value get(Key key) {
        Node x = root;
        while (x != null) {
            int cmp = key.compareTo(x.key);
            if (cmp < 0)
                x = x.left;
            else if (cmp > 0)
                x = x.right;
            else if (cmp == 0)
                return x.val;
        }
        return null;
    }
```

```
56
57      public int size() {
58          return size(root);
59      }
60
61      private int size(Node x) {
62         if (x == null)
63              return 0;
64         return x.count;
65      }
66
67      public Key select(int k) { // returns the nth-largest key
68          if (k < 0)          return null;
69          if (k >= size())    return null;
70          Node x = select(root, k);
71          return x.key;
72      }
73
74      private Node select(Node x, int k) {
75          if (x == null) return (Node) null;
76          int t = size(x.left);
77          if (t > k)
78              return select(x.left, k);
79          else if (t < k)
80              return select(x.right, k - t - 1);
81          else // if (t == k)
82              return x;
83      }
84
85      public void delete(Key key) {
86          root = delete(root, key);
87      }
88
89      private Node delete(Node x, Key key) {
90          if (x == null) return null;
91          int cmp = key.compareTo(x.key);
92          if (cmp < 0) x.left = delete(x.left, key);
93          else if (cmp > 0) x.right = delete(x.right, key);
94          else {
95              if (x.right == null) return x.left;
96              if (x.left == null) return x.right;
97
98              // Update the current node with the minimum value from the
99              // right subtree
100             Node m = min(x.right);
101             x.key = m.key;
102             x.val = m.val;
103             // Remove the minimum node from the right subtree
104             if (x.right == m) {
105                 x.right = m.right;
106             } else {
107                 deleteMinNode(x.right, m);
108             }
109         }
110         x.count = size(x.left) + size(x.right) + 1;
111         return x;
112     }
113
```

```
114     private void deleteMinNode(Node x, Node m) {
115         assert x != m;
116         if (x.left == m) x.left = m.right;
117         else deleteMinNode(x.left, m);
118     }
119 }
```

✎ Übung - Binäre Suchbäume

2.1. Optimale Einfügereihenfolge

Nehmen wir an, dass wir im Voraus schätzen können, wie oft auf Suchschlüssel in einem binären Baum zugegriffen wird. Sollten die Schlüssel in wachsender oder fallender Reihenfolge der zu erwartenden Zugriffshäufigkeit in den Baum eingefügt werden? Warum?

2.2. Minimum bestimmen

Implementieren Sie in BST.java die Methode `public Value min() {...}`, die den Wert mit dem kleinsten Schlüssel zurückgibt, als rekursive Methode.

✎ Übung - Traversierung von binären Suchbäumen

2.3. Rückgabe in absteigender Reihenfolge

Implementieren Sie eine Methode `public Iterable<Key> descending(){... }`, die die Schlüssel in umgekehrter Reihenfolge zurückgibt (d. h. der größte Schlüssel zuerst, der kleinste Schlüssel zuletzt). Implementieren Sie diese Methode rekursiv.

◆ Bemerkung

Es ist eine gute Hausübung die Methode auch einmal nicht-rekursiv zu implementieren.

3. Prioritätswarteschlangen

Vorrangwarteschlange / Priority Queue

Prioritätswarteschlangen - Überblick

- Eine häufige Anforderung ist, dass wir Datensätze mit Schlüsseln der Reihe nach verarbeiten möchten. Wobei der Schlüssel den Prioritätswert für die Reihenfolge der Verarbeitung repräsentiert. Die Verarbeitungsreihenfolge erfolgt dann von der größten zu der kleinsten Priorität oder umgekehrt.
- Anwendungsbeispiele:
  - Job-Scheduling (Schlüssel repräsentieren die Priorität)
  - Heapsort
  - Huffman-Codierung

Prioritätswarteschlangen - Abstrakte Definition

Standardoperationen:

- erzeugen (einer leeren Prioritätswarteschlange)
- einfügen eines neuen Elements mit einer bestimmten Priorität
- abfragen des Elements mit der höchsten Priorität

  (oder ggf. der niedrigsten Priorität je nach Typ der Warteschlange)
- entfernen des Elements mit der höchsten Priorität

  (oder ggf. der niedrigsten Priorität je nach Typ der Warteschlange)

◆ Bemerkung

Eine *Prioritätswarteschlange* (*Priority Queue*) ist somit eine Verallgemeinerung von Stacks und Queues. Beides lässt sich mit Hilfe von Prioritätswarteschlangen implementieren, wenn man als Schlüssel den Einfügezeitpunkt verwendet.

Prioritätswarteschlangen - Implementierung

Verschiedene Implementierungen sind denkbar:

- basierend auf einer (unsortierten) Liste
- basierend auf einem sortierten Array
- basierend auf einem Heap (als nächstes)

Hinweis

Jede Implementierung hat andere Abwägungen und Einsatzszenarien.

Die Verwendung eines Heaps ist jedoch effizient für viele Anwendungen, da das Einfügen und das Entfernen des höchsten Elements in $O(\log n)$ Zeit erfolgen.

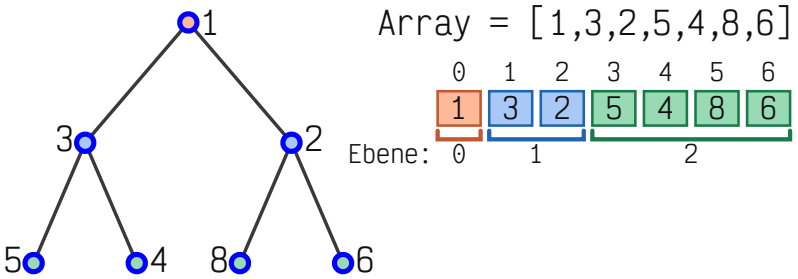
Heap

- Binärbaum, bei dem das Elternelement immer größer oder gleich seinen Kindern ist (Max-Heap).

  (Bei Min-Heaps ist das Elternelement immer kleiner oder gleich seinen Kindern.)
- basierend auf einem fast *vollständigen* Binärbaum (alle Ebenen bis auf die letzte sind vollständig besetzt)
- aus effizienzgründen in einem Array gespeichert

  (Von einem Element `i` aus kann auf das Elternelement bzw. die Kinder über die Index-Formel: `parent = (i-1)/2` und `children = 2i+1, 2i+2` zugegriffen werden, wenn der Wurzelknoten im Array an Position `0` (d. h. Index 0) gespeichert ist.)

Heap - Visualisierung (Min-Heap)



Bemerkung

Die Standardoperationen für Prioritätswarteschlangen sind auf Heaps in $O(\log n)$ Zeit möglich - mit Ausnahme der Abfrage des höchsten Elements, die in $O(1)$ Zeit möglich ist.

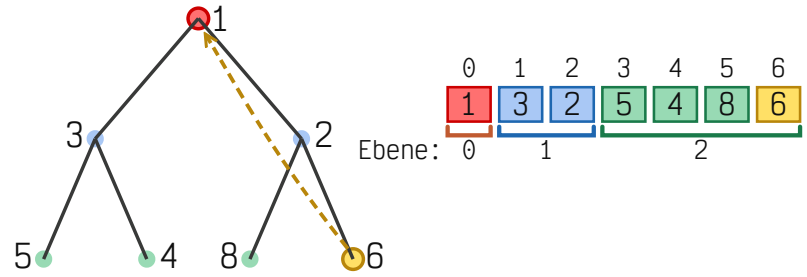
Einfügen eines neuen Elements

Pseudocode-Implementierung

```
// heap - Array, dass die Heap-Eigenschaft erfüllt/den Heap repräsentiert
//        (Das Wurzelelement ist im Array an Position 0 gespeichert.)
// n - Anzahl der Elemente im Heap
// key - der neue Schlüssel
procedure insert(<in/out> heap, <in/out> n, <in> key):
    heap[n] ← key   // Element ans Ende anhängen
    i ← n           // Index des aktuell betrachteten Elements
    n ← n + 1       // Aktualisierung der Anzahl der Elemente

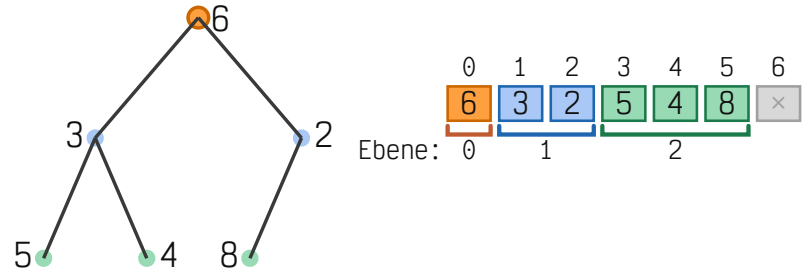
    // up-heap: solange Elternelement kleiner als eingefügtes Element
    while i > 0 and heap[(i-1)/2] < heap[i]:
        swap(heap[(i-1)/2], heap[i])
        i ← (i-1)/2
```

Heap - Entfernen des höchsten Elements (Wert: 1)



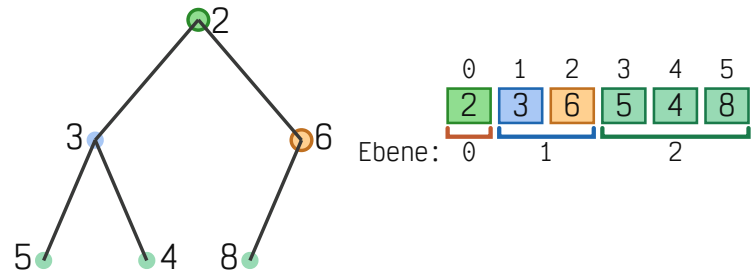
Schritt 1:

- Minimum entfernen.
- Letztes Element (6) wird zur neuen Wurzel.



Schritt 2:

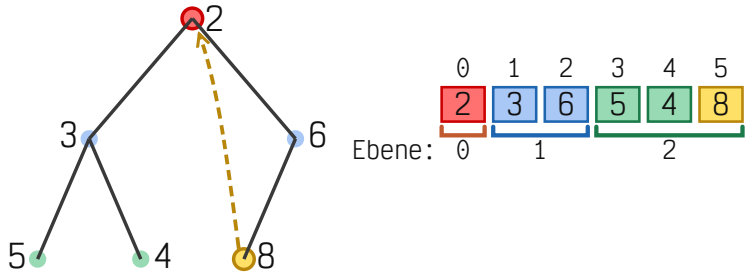
- n = 6 (letzter Slot frei).
- Heap-Eigenschaft verletzt (6 > 3 und 6 > 2).



Schritt 3: Down-heap.

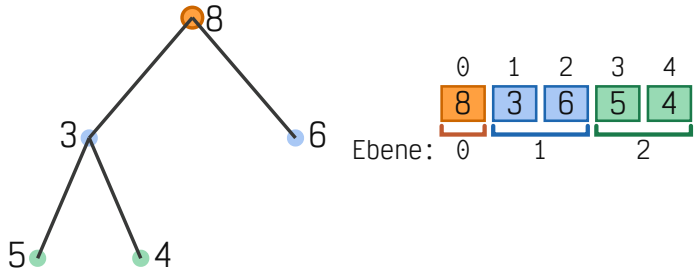
- Tausch 6 ↔ 2 (Index 0↔2).
- 6 ≤ 8: kein weiterer Tausch.
- Heap wiederhergestellt.

Heap - Entfernen des höchsten Elements (Wert: 2)



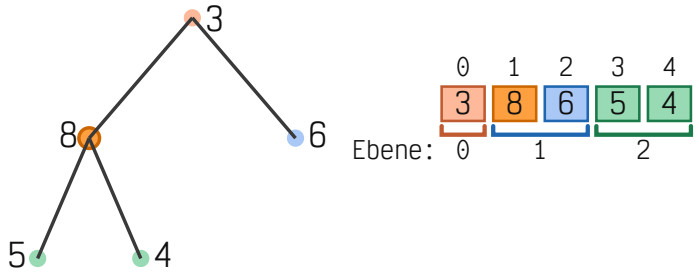
Schritt 1:

- Minimum (2) entfernen.
- Letztes Element (8) wird neue Wurzel.



Schritt 2:

- n = 5. Wurzel = 8.
- Verletzt: 8 > 3 und 8 > 6.

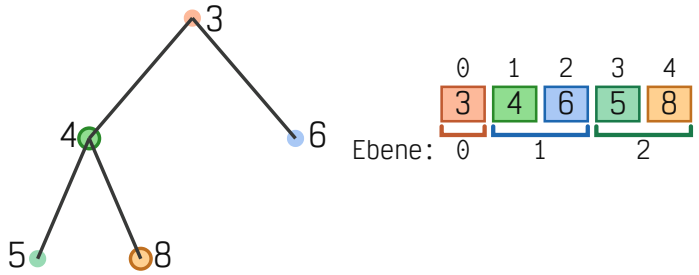


Schritt 3: Down-heap.

- Tausche 8↔3 (Index 0↔1).
- 8 bei Index 1, Kinder: 5 (Index 3), 4 (Index 4).
- 8 > 4 → weiterer Tausch.

※ Hinweis

Ein Tausch von 8 mit 5 hätte die Heapeigenschaft verletzt, da 5 > 4. Es muss immer das Minimum (bei Max-Heaps das Maximum) der Kinder mit dem Elternknoten getauscht werden.



Schritt 4: Down-heap.

- Tausche 8↔4 (Index 1↔4).
- 8 ist Blatt (Index 4).
- Der Heap ist wiederhergestellt.

✎ Übung - Heap

3.1. Basismethoden von Heaps

Implementieren Sie einen Min-Heap. Nehmen Sie das folgende Template als Ausgangsbasis:

1. Implementieren Sie zuerst die Methoden `insert` und `isEmpty/nonEmpty`. Fügen Sie Ihrer Implementierung eine Möglichkeit hinzu den internen Zustand für Debugzwecke auszugeben.
   (Gehen Sie davon aus, dass die Kapazität des Arrays nie überschritten wird.)
2. Überlegen Sie sich noch einmal genau welche Schritte Sie in Ihrer Implementierung durchführen müssen, um ein Element zu entfernen (⇒ `remove`). Danach implementieren Sie die Methode `remove`.
3. Testen Sie Ihre Implementierung.

```
class Heap<T extends Comparable<T>> {

    // Min-heap!

    private T[] heap;
    private int size;

    @SuppressWarnings("unchecked")
    public Heap(Class<T> componentType, int capacity) {
        heap = (T[]) Array.newInstance(componentType, capacity);
        size = 0;
    }

    // TODO nonEmpty

    // TODO insert

    public T remove() {
        throw new UnsupportedOperationException("remove is not implemented yet");
    }

    @SafeVarargs
    final void insertAll(T... args) {
        for (T arg : args) {
            insert(arg);
        }
    }
}

void main() {
    Heap<String> heap = new Heap<>(String.class, 5);
    heap.insertAll("dies", "ist", "ein", "wichtiger", "test");
    while (heap.nonEmpty()) {
        String s = heap.remove();
        IO.println(s);
    }
}
```

✎ Übung - Heap zum Sortieren

3.2. Sortierte Liste

Implementieren Sie eine generische, statische Methode `sort` auf Ihrer Heapklasse aus der vorhergehenden Übung. Die methode `sort` soll eine passende typisierte `java.util.Collection` übernehmen und diese als sortierte Liste zurückgeben.

Achten Sie auf die korrekte Verwendung der generischen Typparameter. Beachten Sie, dass Sie für den Aufruf des Heap-Konstruktors den `elementType` angeben müssen. Weiterhin soll es möglich sein, dass der Typ der Elemente der übergebenen Collection ein Subtype des `elementTypes` ist.

Nutzen Sie die folgende `main` Methode, um Ihre Implementierung zu testen. Nehmen Sie keine Änderungen an dieser Methode vor.

```
public static void main(String[] args) {
    var sorted = Heap.sort(Integer.class, List.of(3, 5, 6, 2, 4, 5, 1, 2, 4));
    IO.println(sorted);

    List<java.sql.Date> dates = List.of(
        new java.sql.Date(242342342342341),
        new java.sql.Date(109480934831)
    );
    var sortedDates = Heap.<java.util.Date>sort(java.util.Date.class, dates);
    IO.println(sortedDates);
}
```